

RELATÓRIO TÉCNICO

Priorização de tickets de suporte técnico com lógica fuzzy

Disciplina: Inteligência Artificial e Computacional

Tema: Sistemas de Controle Fuzzy

Gabriel Albuquerque

Integrantes
Davi Maciel

Alberto Acosta

Resumo

Este trabalho apresenta um protótipo mínimo e reproduzível de priorização de tickets. A solução usa um sistema fuzzy Mamdani com três entradas, uma saída, doze regras, visualização das funções de pertinência, cenários de teste e análise de sensibilidade.

GitHub: <https://github.com/Gaalbu/Fuzzy-sistem>

Entrega acadêmica preparada para o repositório GitHub do projeto.

1. Problema e justificativa

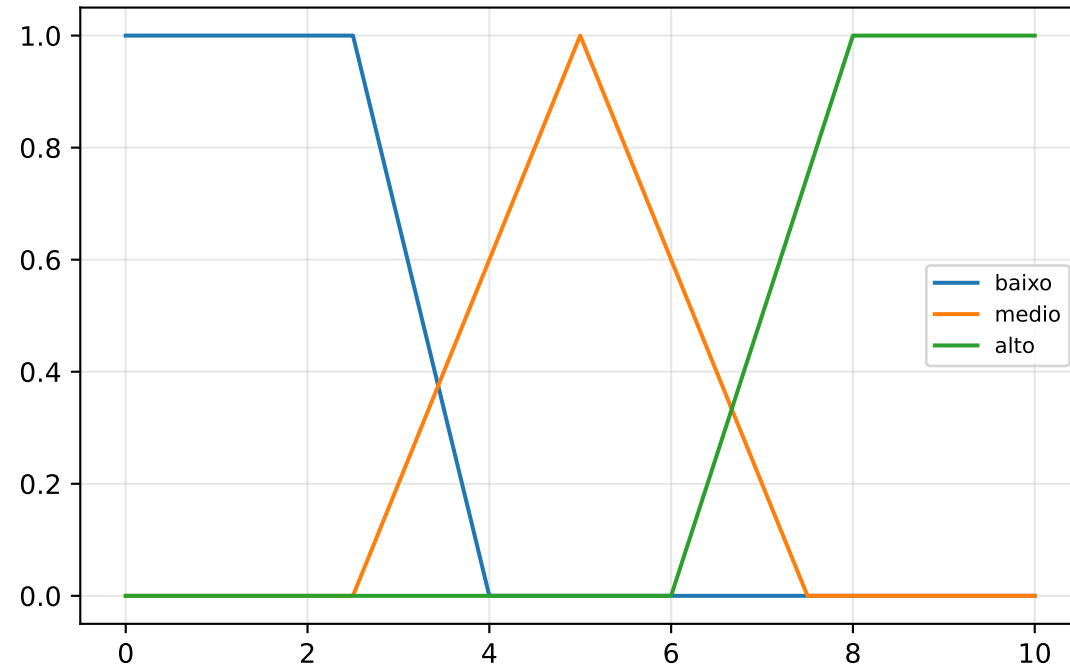
Problema: definir, de forma transparente e consistente, a prioridade de atendimento de tickets de suporte com informações parcialmente subjetivas. A lógica fuzzy é adequada porque o decisor trabalha com termos linguísticos como baixo, médio, alto e crítico. Modalidade escolhida: Opção B, produto/protótipo. Requisitos mínimos atendidos: 3 entradas, 1 saída, 12 regras efetivas, 6 cenários de teste, gráficos, manual de execução, código organizado e declaração de uso de IA.

2. Extensão opcional escolhida

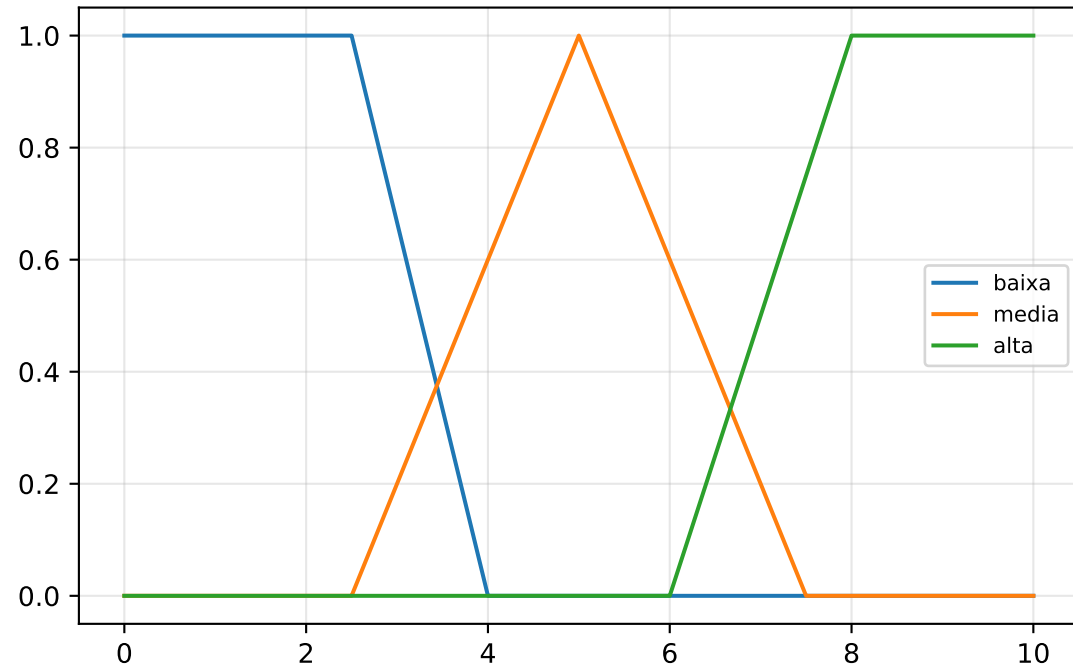
Foi adotado o caminho mais simples da pontuação extra: levantamento bibliográfico complementar com bases reconhecidas e autores clássicos da área fuzzy, sem aumentar a complexidade do código principal. Bases/termos de busca: Google Scholar, IEEE Xplore e Scopus; fuzzy control, Mamdani, TSK, fuzzy rule base. Referências-base para esse reforço: Zadeh (1965), Mamdani & Assilian (1975), Ross (2010) e Klir & Yuan (1995).

3. Funções de pertinência

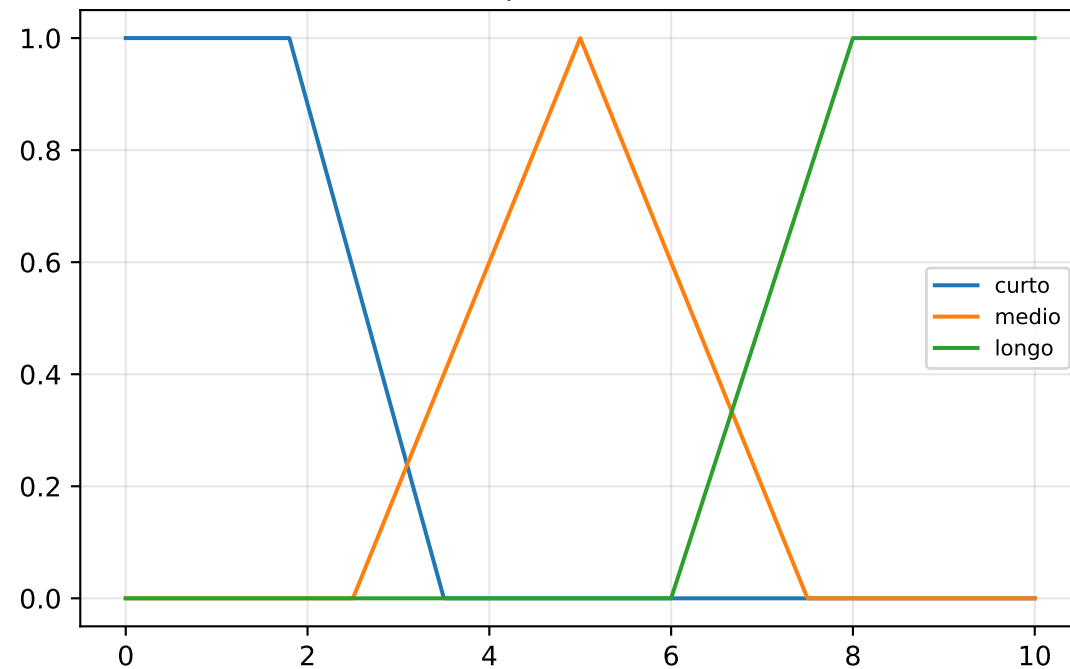
Impacto



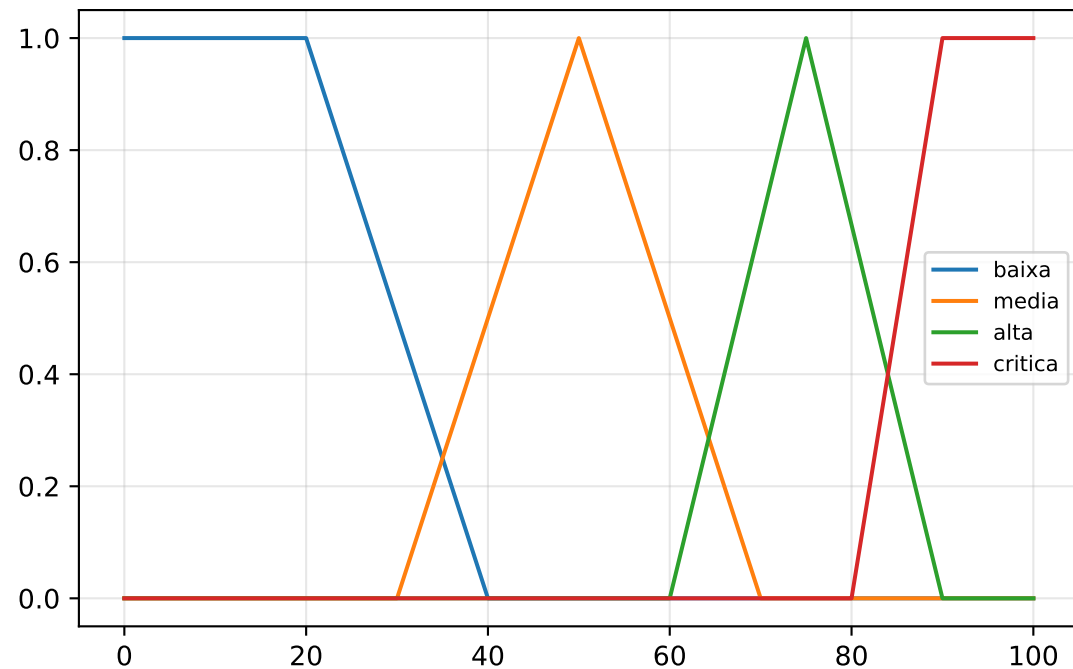
Urgência



Tempo em aberto



Prioridade



4. Base de regras

#	Impacto	Urgência	Tempo	Saída
1	alto	alta	longo	critica
2	alto	alta	curto	alta
3	alto	media	longo	alta
4	alto	baixa	longo	alta
5	medio	alta	longo	alta
6	medio	alta	medio	alta
7	medio	media	medio	media
8	medio	baixa	curto	media
9	baixo	alta	longo	media
10	baixo	media	medio	media
11	baixo	baixa	longo	baixa
12	baixo	baixa	curto	baixa

Inferência: operador AND = mínimo; agregação = máximo; defuzzificação = centróide.

5. Implementação e organização

Arquitetura: um único módulo de regra e um script de geração de artefatos. O objetivo foi manter o código curto, legível e fácil de reproduzir. Estrutura do repositório: - src/fuzzy_system.py - src/build_artifacts.py - docs/manual_execucao.md - docs/declaracao_ia.md - docs/roteiro_gamma.md - artifacts/relatorio_fuzzy.pdf - artifacts/apresentacao_fuzzy.pdf

Decisão técnica

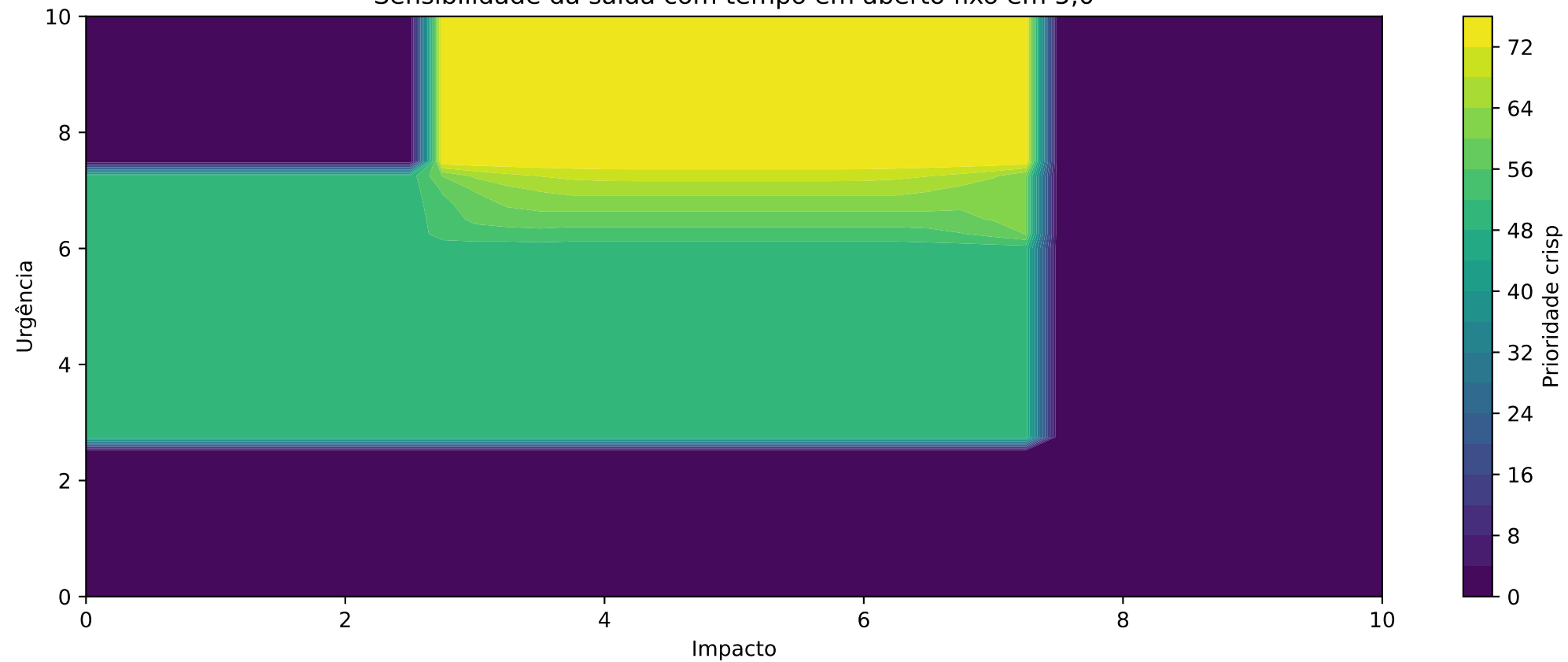
Foi evitada qualquer dependência pesada: apenas numpy e matplotlib.

6. Cenários de teste

Impacto	Urgência	Tempo	Saída	Classe	Esperado
1.0	1.0	1.0	15.56	baixa	baixa
3.0	4.0	3.0	50.00	media	media
5.0	5.0	5.0	50.00	media	media
8.5	8.5	9.5	92.22	critica	critica
9.0	2.0	7.5	75.00	alta	alta
2.5	9.0	8.0	50.00	media	media

Os cenários cobrem casos baixos, médios, altos, fronteiriços e conflitantes.

Sensibilidade da saída com tempo em aberto fixo em 5,0



7. Conclusão, IA e referências

Conclusão: o protótipo atende ao escopo acadêmico, gera saída contínua, é reproduzível e permite defesa técnica. Como limitações, a base de regras pode ser ampliada e a validação pode ser aprofundada com dados reais. Declaração de IA: GPT-5.5 Pro da OpenAI foi usado como apoio na escrita, organização e revisão do código. O material final foi conferido manualmente antes da exportação. Referências-base: Zadeh (1965), Mamdani & Assilian (1975), Ross (2010), Klir & Yuan (1995).